



Primer Estudio de Caso.

Análisis de Expresiones Aritméticas

Dylan Josue Chaves Hernández

Universidad CENFOTEC

Estructuras de Datos

Romario Salas Cerdas

Fecha: 10, 2025

Resumen Ejecutivo

El presente trabajo de investigación explora la estructura de datos de pila y su aplicación fundamental en el campo de la ciencia de la computación, específicamente en el análisis y evaluación de expresiones aritméticas. Se detalla el principio LIFO (Last-In, First-Out) el cual es el comportamiento de las pilas y se argumenta por qué esta característica las convierte en la herramienta ideal para gestionar la jerarquía de operadores y el anidamiento de paréntesis.

Se analiza la eficiencia, seguridad y el orden que esta estructura aporta al proceso. Finalmente, se presenta un caso de estudio concreto sobre la Máquina Virtual de Java (JVM), demostrando cómo un software de sistema de alto impacto utiliza una arquitectura basada en pilas para ejecutar cálculos aritméticos de manera sistemática y eficiente.

Tabla de Contenido

Introducción.....	4
Desarrollo.....	5
La Pila como Estructura de Datos.....	5
Relación entre Pilas y Evaluación de Expresiones Aritméticas.....	5
Conversión de Infijo a Postfijo.....	5
Evaluación de una Expresión Postfija con Pila.....	6
Ventajas de la Pila como estructura de Datos.....	7
Aplicación en Software de Sistema.....	8
Representación Gráfica del Proceso.....	9
Conclusión.....	11
Referencias.....	12

Introducción

En el desarrollo de software, la elección de la estructura de datos adecuada es fundamental para la creación de algoritmos eficientes y robustos. Una de las estructuras más elementales e importantes es la pila. Aunque su concepto es simple, sus aplicaciones son vastas y críticas para el funcionamiento de sistemas complejos, desde la gestión de llamadas a funciones en la memoria de un programa hasta la funcionalidad de "deshacer" en editores de texto.

El objetivo de esta investigación es analizar en profundidad el rol de la pila en una tarea computacional clásica: el análisis de expresiones aritméticas. Los compiladores e intérpretes de lenguajes de programación no procesan las operaciones matemáticas en el formato infijo al que los humanos están acostumbrados. En su lugar, dependen de algoritmos que utilizan pilas para transformar estas expresiones a notaciones más simples (como la postfija) para posteriormente, evaluarlas.

Este documento demostrará cómo el principio LIFO de la pila se alinea de manera natural con las reglas de precedencia de operadores, ofreciendo una solución ordenada, segura y de alto rendimiento.

Desarrollo

La Pila como Estructura de Datos

Una pila es una estructura de datos lineal que se define por su modo de acceso a los datos, basado en el principio LIFO (Last-In, First-Out). Esto implica que el último elemento que ingrese a la colección es el primer elemento que se puede retirar. Se puede visualizar como una pila de sillas: solo se puede interactuar con la silla que se encuentra en el tope.

Las operaciones fundamentales de una pila son:

- Push (Apilar): Añade un nuevo elemento en la cima (o tope) de la pila.
- Pop (Desapilar): Elimina y devuelve el elemento que se encuentra en la cima de la pila.

Adicionalmente, se suelen implementar operaciones auxiliares como:

- Peek (Mirar): Consultar el elemento de la cima sin retirarlo).
- isEmpty (Está Vacío): Verificar si la pila está vacía.

Relación entre Pilas y Evaluación de Expresiones Aritméticas

Cuando se tiene una expresión aritmética en notación infija (por ejemplo, $3 + 4 * (2 - 1)$), el reto es que los operadores tienen distinta precedencia y que los paréntesis cambian el orden de evaluación. La pila resulta una estructura muy natural para manejar estos retos: sirve como memoria “temporal” para los operadores mientras se decide cuándo aplicarlos.

Conversión de Infijo a Postfijo

Uno de los usos más clásicos es usar el algoritmo shunting yard de Dijkstra para convertir una expresión infija a postfija, usando una pila para operadores.

La sucesión de pasos que ejecuta dicho algoritmo es al siguiente:

1. Recorrer la expresión infija de izquierda a derecha.
2. Si encuentra un operando (número, variable), lo agrega directamente a la salida (la expresión postfija en construcción).
3. Si encuentra un paréntesis izquierdo "(", lo apila (push) en la pila de operadores.
4. Si encuentra un paréntesis derecho ")", desapilar (pop) operadores de la pila hasta encontrar el paréntesis izquierdo correspondiente, y esos operadores van a la salida.
5. Si encuentra un operador op, se hace lo siguiente:
 - Mientras la pila no esté vacía y el operador en la cima tenga mayor precedencia que op, o la misma precedencia (y op sea asociativo izquierda), desapilar ese operador y lo envía a la salida.
 - Luego apila op en la pila de operadores.
6. Al final del recorrido, desapila todos los operadores restantes y los agrega a la salida.

Este algoritmo asegura que se respete la precedencia y la asociatividad de los operadores. La complejidad es $O(n)$ siendo "n" el número de tokens, porque cada token se procesa en tiempo constante promedio y cada operador entra y sale de la pila como mucho una vez. Una vez que tienes la expresión en forma postfija (RPN), la evaluación es más directa.

Evaluación de una Expresión Postfija con Pila

Para evaluar una expresión en postfijo (por ejemplo $5\ 6\ 2\ +\ *\ 12\ 4\ /\ -$), se usa otra pila (pila de operandos) y se ejecutan los siguientes pasos:

1. Recorrer la expresión postfija de izquierda a derecha.
2. Si el token es un operando (por ejemplo "5"), se convierte a valor numérico y lo apila.
3. Si el token es un operador (por ejemplo *), hacer:
 - $op2 = pop()$
 - $op1 = pop()$
 - $resultado = aplicar\ operador(op1, op2)$
 - $push(resultado)$

4. Al final del recorrido, debe quedar exactamente un valor en la pila: ese es el resultado final.

Este método es eficiente ($O(n)$) y correcto siempre que la expresión postfija esté bien formada.

Algunos autores combinan la conversión y evaluación en una sola pasada “two-stack algorithm” (una pila para operandos y otra para operadores) para evaluar directamente la expresión infija sin convertir primero explícitamente a postfijo.

Ventajas de la Pila como estructura de Datos

Orden

La naturaleza LIFO impone una secuencia de procesamiento determinista y rigurosa. No hay lugar para la ambigüedad; cada token (número u operador) se trata según un conjunto de reglas claras, garantizando que la evaluación sea consistente y correcta.

Eficiencia

Las operaciones push y pop son de tiempo constante, es decir, su complejidad algorítmica es $O(1)$. Esto significa que el tiempo de ejecución no depende del número de elementos en la pila. Como resultado, los algoritmos que utilizan pilas para analizar expresiones, como el algoritmo Shunting-yard, tienen una complejidad lineal $O(n)$, donde n es el número de tokens en la expresión. Esto los hace extremadamente eficientes.

Seguridad

El acceso restringido (solo al tope) minimiza la posibilidad de errores de programación, como la corrupción de datos o el acceso a índices inválidos, que podrían ocurrir con estructuras de acceso aleatorio como los arreglos.

Por lo tanto, frente a otras estructuras como listas enlazadas o colas, la pila impone, por defecto, restricciones que favorecen la lógica de evaluación de expresiones, con poca sobrecarga.

Aplicación en Software de Sistema

Un ejemplo del uso de pilas se encuentra en la arquitectura de la Máquina Virtual de Java (JVM). La JVM no opera directamente sobre registros de la CPU, sino que es una máquina de pila.

Cuando un programa Java es compilado, el código fuente se convierte en un código intermedio llamado bytecode. Para ejecutar este bytecode, la JVM utiliza un área de memoria para cada hilo de ejecución, la cual contiene una pila de operandos (operand stack). Cada método invocado tiene su propio espacio (un frame), y dentro de ese espacio, la pila de operandos se usa para realizar todos los cálculos.

El compilador javac la traduce a un bytecode que manipula la pila de operandos. Usando la herramienta javap -c, se puede observar las siguientes instrucciones:

- `iconst_3`: Push 3 en la pila
- `iconst_5`: Push 5 en la pila
- `iadd`: Pop 5 y 3, calcula la suma (8), y Push el resultado (8)
- `iconst_2`: Push 2 en la pila
- `imul`: Pop 2 y 8, calcula el producto (16), y Push el resultado (16)
- `istore_1`: Pop el resultado final (16) y lo almacena en la variable 'resultado'

Este ejemplo ilustra cómo la JVM descompone una expresión compleja en una secuencia de operaciones atómicas y seguras, todas orquestadas a través de una pila.

Representación Gráfica del Proceso

Para ilustrar el flujo completo, analizaremos la expresión $5 + (6 * 2)$.

Conversión de Infijo a Postfijo

Se utiliza una pila para almacenar temporalmente los operadores.

Token de Entrada	Pila de Operadores	Salida Postfija
5	Vacía	5
+	+	5
(	+ (	5
6	+ (	5 6
*	+ (*	5 6
2	+ (*	5 6 2
)	+	5 6 2 *
Fin	Vacía	5 6 2 * +

Evaluación de la Expresión Postfija

Se utiliza una pila para almacenar los operandos (números) y los resultados intermedios.

Token de Entrada	Estado de la Pila	Descripción
5	[5]	Push 5.
6	[5, 6]	Push 6.
2	[5, 6, 2]	Push 2.
*	[5, 12]	Pop 2 y 6, calcula $6 * 2 = 12$, Push 12.
+	[17]	Pop 12 y 5, calcula $5 + 12 = 17$, Push 17.

Conclusión

La estructura de datos de pila, a pesar de su simplicidad conceptual, es una herramienta indispensable en la ciencia de la computación. Su principio fundamental LIFO proporciona una solución elegante, eficiente y segura para el problema del análisis de expresiones aritméticas. Al permitir el aplazamiento y la priorización de operaciones de manera sistemática, las pilas forman la base sobre la cual los compiladores e intérpretes modernos, como la Máquina Virtual de Java, son capaces de traducir la lógica matemática legible por humanos a un formato ejecutable por máquinas.

La comprensión de este mecanismo no solo es crucial para el estudio de algoritmos y estructuras de datos, sino que también ofrece una visión clara del funcionamiento interno de las tecnologías de software que utilizamos a diario.

Referencias

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). Compilers: Principles, Techniques, and Tools (2nd ed.). Addison-Wesley.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

Lindholm, T., Yellin, F., Bracha, G., & Buckley, A. (2015). The Java Virtual Machine Specification, Java SE 8 Edition. Oracle Corporation.